

KECDrives — Storage & VM Estimate (60K Students)

Date: 27 March 2026

Scope: PostgreSQL, Garage (S3-compatible object storage), Docker containers, monitoring stack

Target: 60,000 students (3,000/year × 5+ years), 300+ companies/year, 3,000 student records/year

Current VM: Ubuntu 24.04, 500 GB SSD, `sda3` 496.9G partition (383 GB used, 82 GB free)

0. Current VM Status & Garage Partition Issue

Problem: Garage is allocated ~350 GiB of the 500 GB disk, leaving only ~82 GB for the rest (Postgres, Docker, OS, monitoring). This is over-provisioned for current usage and squeezing Postgres/Docker.

Recommended fix: Re-partition or configure Garage's `data_dir` to a specific directory with disk quotas, or resize the VM to appropriately split storage. See Section 7 for new VM sizing.

1. PostgreSQL Database Storage

Per-Student Row Sizes

Table	Columns	Est. Row Size	Rows per Student
<code>users</code>	14 cols (email, password_hash, name, etc.)	~400 B	1
<code>student_personal</code>	~25 cols (register_number, dob, address, pan_number, aadhar_number, etc.)	~500 B	1
<code>student_schooling</code>	~16 cols (marks, boards, institutions)	~350 B	1
<code>student_degrees</code>	~10 cols (cgpa, semester_gpas JSONB, university)	~300 B	2 (UG + PG)
<code>student_documents</code>	resume_url, timestamps (aadhar/pan URLs to be removed)	~150 B	1
<code>student_change_requests</code>	edit approval flow	~200 B	~3 avg

Per student total: ~400 + 500 + 350 + (300 × 2) + 150 + (200 × 3) = **2,600 B ≈ 2.5 KB**

60,000 Students

Item	Calculation	Size
Student data (rows)	60,000 × 2.5 KB	150 MB
Indexes (B-tree overhead ~30%)	150 MB × 0.30	45 MB
Student DB subtotal		~195 MB

Per-Drive (Company) Row Sizes

Table	Est. Row Size	Rows per Drive
placement_drives	~1.2 KB (includes JSONB rounds, attachments)	1
job_roles	~120 B	3 avg
drive_eligible_departments	~30 B	5 avg
drive_eligible_batches	~20 B	2 avg
drive_applications	~120 B	varies

Per drive total: $\sim 1,200 + (120 \times 3) + (30 \times 5) + (20 \times 2) = 1,750 \text{ B} \approx 1.7 \text{ KB}$ (before applications)

500 Drives/Year $\times$ Applications

Assuming 3,000 student applications per year (spread across 500 drives):

Item	Calculation	Size
Drives (500/yr)	$500 \times 1.7 \text{ KB}$	0.85 MB
Applications (3,000/yr)	$3,000 \times 120 \text{ B}$	0.36 MB
Drive data per year		~1.5 MB
10 years accumulated	$1.5 \text{ MB} \times 10$	~15 MB

Other Tables

Table	Estimate
chat_messages (if used heavily)	$\sim 500 \text{ B} \times 500\text{K messages} = \textbf{250 MB}$
chat_groups + chat_group_members	~5 MB
activity_logs (audit trail)	$\sim 500 \text{ B} \times 1\text{M entries} = \textbf{500 MB}$
admin tables (spocs, settings, templates)	< 1 MB
auth tables (password_resets)	< 1 MB

PostgreSQL Total Estimate

Component	Size
Student data + indexes	195 MB
Drive data (10 years)	15 MB
Chat data	250 MB
Activity logs	500 MB

Component	Size
Other tables	5 MB
WAL + temp space + VACUUM overhead (~50%)	485 MB
PostgreSQL Total	~1.5 GB

Recommendation: Allocate **5 GB** for PostgreSQL data volume to allow comfortable growth, WAL logs, temp tables, and pg_dump backups.

2. Garage (S3-Compatible Object Storage)

Per-Student Documents (After Cleanup)

With aadhar/pan card **documents removed** (only storing numbers in DB):

Document	Max Size	Avg Size Est.
Profile Photo	1 MiB limit	~200 KB avg
Resume (PDF)	1 MiB limit	~500 KB avg
Per student	2 MiB max	~700 KB avg

60,000 Students

Item	Calculation	Size
Best case (all upload, avg size)	60,000 × 700 KB	42 GB
Worst case (all at 1 MiB each)	60,000 × 2 MiB	120 GB
Realistic (70% upload rate, avg)	42,000 × 700 KB	~30 GB

Drive Attachments (Company Data)

Item	Per Drive	Per Year (500 drives)	10 Years
Attachments (JD PDFs, etc.)	5 MiB max	500 × 5 MiB = 2.5 GB	25 GB
Company Logos	~50 KB avg	500 × 50 KB = 25 MB	250 MB

Chat Attachments

Item	Estimate
Chat files (if used)	~5–10 GB over time

Garage Metadata Overhead

Garage stores metadata (SQLite) alongside data. Expect ~5–10% overhead.

Garage Total Estimate

Component	Conservative	Worst Case
Student docs (60K)	30 GB	120 GB
Drive attachments (10 yrs)	25 GB	25 GB
Company logos	0.25 GB	0.25 GB
Chat attachments	10 GB	10 GB
Metadata overhead (~10%)	6.5 GB	15.5 GB
Garage Total	~72 GB	~171 GB

Garage 5-Year Growth Projection

Year	New Students	Cumulative Students	Student Docs (avg)	Drive Attachments	Cumulative Garage
1	3,000	3,000	2.1 GB	1.5 GB	~4 GB
2	3,000	6,000	4.2 GB	3.0 GB	~8 GB
3	3,000	9,000	6.3 GB	4.5 GB	~12 GB
4	3,000	12,000	8.4 GB	6.0 GB	~16 GB
5	3,000	15,000	10.5 GB	7.5 GB	~20 GB
Legacy (pre-existing 45K)	—	60,000	30 GB	—	~50 GB

At 5 years with 15K new students + 45K legacy imports, realistic Garage usage is **50–80 GB**.

Recommendation: Allocate **100 GB** for Garage storage (realistic scenario). Scale to 200 GB if worst-case growth expected.

3. Docker Containers & System

Container Images + Runtime

Container	Image Size	Runtime Memory
auth-service	~20 MB	~30 MB
drive-service	~25 MB	~50 MB
student-service	~25 MB	~50 MB
admin-service	~25 MB	~50 MB
analytics-service	~20 MB	~30 MB

Container	Image Size	Runtime Memory
chat-service	~20 MB	~50 MB
PostgreSQL 15	~230 MB	~512 MB (shared_buffers=256MB)
Redis 7	~30 MB	~100 MB
Garage (S3)	~50 MB	~100 MB
Caddy (reverse proxy)	~40 MB	~30 MB
Prometheus	~200 MB	~200 MB
Grafana	~400 MB	~150 MB
pgAdmin	~400 MB	~200 MB
Node Exporter	~20 MB	~10 MB
cAdvisor	~80 MB	~50 MB
Postgres Exporter	~20 MB	~10 MB
Redis Exporter	~15 MB	~10 MB
Dozzle (logs)	~20 MB	~30 MB
Totals	~1.6 GB	~1.7 GB

Docker Volumes (non-data)

Volume	Size
caddy_data + caddy_config	~50 MB
prometheus_data (15d retention)	~2 GB
grafana_data	~200 MB
pgadmin_data	~100 MB
Docker overlay/layers	~3 GB
Subtotal	~5.5 GB

OS + Docker Engine

Component	Size
Linux base (Ubuntu/Debian minimal)	~2 GB
Docker Engine + dependencies	~1 GB
System logs, tmp, swap	~2 GB
Subtotal	~5 GB

4. Total VM Storage Estimate

Component	Recommended	Minimum
PostgreSQL data	5 GB	2 GB
Garage object storage	100 GB	75 GB
Docker images + layers	5 GB	3 GB
Monitoring data	3 GB	2 GB
OS + Docker Engine	5 GB	4 GB
Headroom (20%)	24 GB	17 GB
TOTAL	~142 GB	~103 GB

Recommended VM Spec

Resource	Recommendation
Storage	150 GB SSD (comfortable headroom for 60K students + 10 years of drives)
RAM	4 GB minimum (all containers + Postgres shared_buffers=256MB + Redis)
CPU	2 vCPUs minimum (4 recommended for concurrent queries)

If chat is heavily used or you expect >100K students, scale to **200 GB SSD** and **8 GB RAM**.

5. Performance Notes

Issues Identified & Fixed

Issue	Impact	Fix
<code>TriggerAutoStatusUpdates()</code> runs 2 UPDATES on every <code>GetDrives()</code> call	Slow writes on every list request	Added partial indexes on <code>status + deadline_date</code> and <code>status + drive_date</code>
No indexes on <code>drive_applications(drive_id)</code> , <code>job_roles(drive_id)</code>	Correlated subqueries do full table scans	Added indexes (migration <code>0005_performance_indexes.sql</code>)
No indexes on <code>student_personal(batch_year, department)</code>	Slow student listing/filtering	Added composite index
Duplicate search filter in admin-service <code>GetDrives</code>	Search applied twice, wasted computation	Removed duplicate

Issue	Impact	Fix
<code>UploadProfilePicture</code> allowed 2 MiB	Inconsistent with 1 MiB requirement	Fixed to 1 MiB across all services
Drive attachments had no size limit	Unbounded uploads possible	Added 5 MiB per-file limit
Legacy API still accepted aadhar/pan document uploads	Unnecessary storage of sensitive docs	Removed from valid types

Redis Caching (Already Implemented)

- Drive listings: cached 4 minutes with key pattern `api:admin:drives:*` / `api:student:drives:*`
- Student profiles: cached 5 minutes with key `student:profile:{id}`
- Cache invalidation on write operations 

Recommended Additional Optimizations

1. **Move `TriggerAutoStatusUpdates` to a cron job** (run every 5 min via `pg_cron` or a Go ticker) instead of on every request
2. **Add `pg_trgm` extension** for the GIN index on `company_name` to work: `CREATE EXTENSION IF NOT EXISTS pg_trgm;`
3. **Consider read replicas** if query load exceeds single-instance capacity at 60K+ students

6. Summary

Storage Target	Allocation
PostgreSQL	5 GB
Garage (Object Storage)	100 GB
Docker + Monitoring + OS	15 GB
Headroom	30 GB
VM Disk Total	150 GB SSD
VM RAM	4–8 GB
VM CPU	2–4 vCPUs

For 60,000 students with profile photos + resumes (1 MiB each), 300+ companies/year with 5 MiB attachments, and all services running in Docker — a **150 GB SSD VM with 4 GB RAM** is the sweet spot.

7. New VM Recommendation (5-Year Projection)

Can Our DB Hold 5 Years of Data?

Component	Year 1	Year 3	Year 5	Notes
Students in DB	3,000	9,000	15,000	3,000 added/year
Companies	300	900	1,500	300/year
Applications per year	3,000	3,000	3,000	
PostgreSQL total	~300 MB	~700 MB	~1.2 GB	Including indexes + WAL
Garage total	~4 GB	~12 GB	~20 GB	Student docs + drive attachments

With 60K legacy import: PostgreSQL ~1.5 GB, Garage ~80 GB max.

Answer: Yes, 5 GB PostgreSQL allocation easily holds 5+ years. Garage at 100 GB handles it comfortably.

Recommended New VM Spec

Resource	Minimum	Recommended
Storage	150 GB SSD	200 GB SSD (room for growth + backups)
RAM	4 GB	8 GB (Redis cache + Postgres shared_buffers)
CPU	2 vCPUs	4 vCPUs (concurrent Go services + DB)

Disk Partitioning Recommendation

Partition	Size	Purpose
/ (root)	30 GB	OS, Docker Engine, container images
/var/lib/docker	15 GB	Docker volumes (Postgres, Redis, Prometheus)
/var/lib/garage	120 GB	Garage object storage
Unallocated/backup	35 GB	Headroom + local backup staging
Total	200 GB	

8. Backup Strategy

PostgreSQL Backup

Method	How	Frequency	Storage
pg_dump compressed	pg_dump -Fc dbname > backup.dump	Daily (cron at 2 AM)	~500 MB compressed
Retention	Keep last 7 daily + 4 weekly + 3 monthly	Rolling	~5 GB max
Where to store	Ship to external storage (SCP/rsync to another server, or S3-compatible remote)	After each dump	Off-VM

Example cron:

```
# /etc/cron.d/pg_backup
0 2 * * * root docker exec postgres pg_dump -Fc -U postgres kecdrives >
/backups/pg/kecdrives_$(date +%Y%m%d).dump && find /backups/pg/ -mtime
+7 -delete
```

Garage (Object Storage) Backup

Method	How	Frequency
garage repair + restic	Use restic or rclone to sync Garage data dir to remote	Weekly
Bucket export	Use aws s3 sync (with Garage S3 endpoint) to mirror to external S3	Weekly
Retention	Keep last 4 weekly snapshots	Rolling

Example:

```
# Sync Garage bucket to external backup server using rclone
0 3 * * 0 root rclone sync garage:student-documents remote-
backup:kecdrives-garage-backup --transfers=4
```

Disaster Recovery

Scenario	Recovery
VM disk failure	Restore from latest pg_dump + rclone backup on new VM
Accidental data deletion	Restore specific pg_dump + retrieve objects from Garage backup
VM migration	pg_dump → new VM pg_restore + rclone sync Garage data
RTO (Recovery Time)	~30 minutes for DB, ~2 hours for full Garage restore
RPO (Data Loss)	Max 24 hours (daily DB backup)

9. Architecture Notes

Uptime & Latency

- **Single VM = single point of failure.** If the VM goes down, everything goes down.
- For higher uptime: consider a standby VM with pg_basebackup streaming replication.
- Current architecture is fine for a college placement portal with expected usage patterns (business hours, ~200 concurrent users max).
- **Latency:** All services on same VM = minimal network latency between services. Redis caching reduces DB load.

Redis Cache Effectiveness

Redis is already used for:

- Drive listings: 4-min cache (`api:admin:drives:*`)
- Student profiles: 5-min cache (`student:profile:{id}`)

Additional Redis caching recommendations for slow pages:

- Cache the drive detail page's eligible students list (heavy query with 2000+ students)
- Cache department/batch config (rarely changes, currently fetched every page load)
- Cache dashboard stats (computed aggregates)

Key: Move `TriggerAutoStatusUpdates` to Background

The single biggest performance win: stop running 2 UPDATE queries on every `GetDrives()` call. Run it as a cron job every 5 minutes instead.